

Red Hat Portfolio Architecture v1.9



May 2022



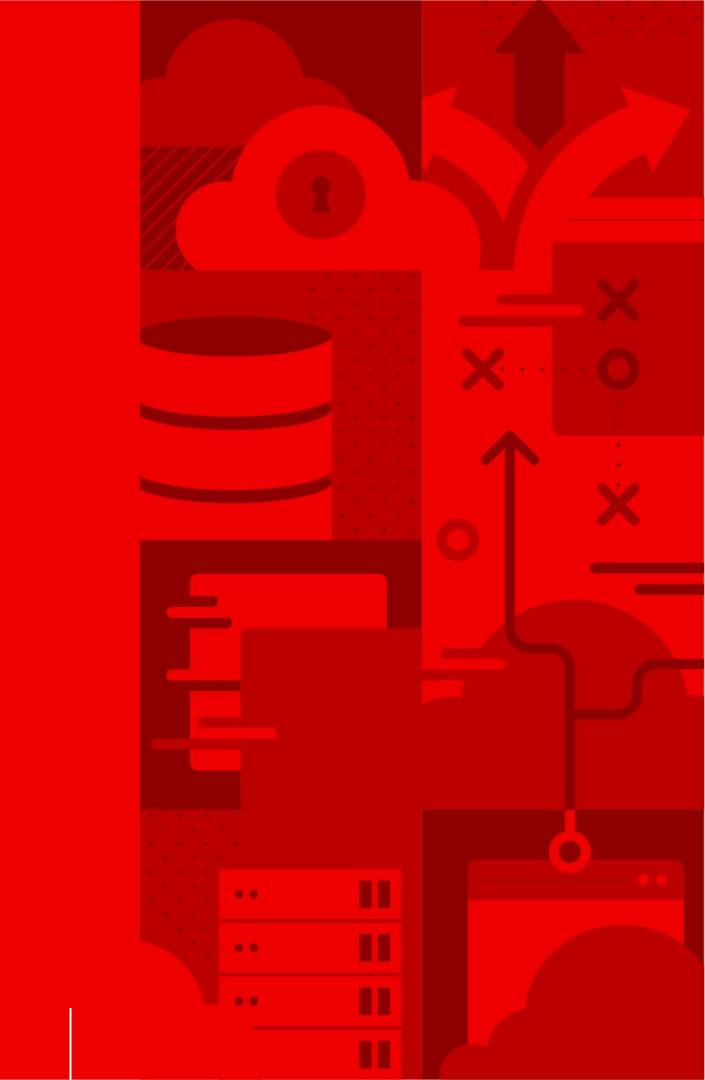


"Cloud-native development is an approach to building and running applications to fully exploit the advantages of the cloud computing model (i.e. responsive, elastic and resilient applications)."



Tackling the challenges

- Workflow for development with local container tooling
- Workflow for development on remote container platform
- A cloud-native development deployment strategy
- Setting up enterprise image registry strategy

A vertical red graphic on the left side of the slide. It contains various white icons representing technology: a cloud with a keyhole, a database cylinder, a server rack, a monitor, and several arrows pointing in different directions. There are also 'X' and 'O' symbols and a circuit-like line.

Portfolio Architecture

Generic view of the architecture

Cloud Native Development Introduction

Business Drivers



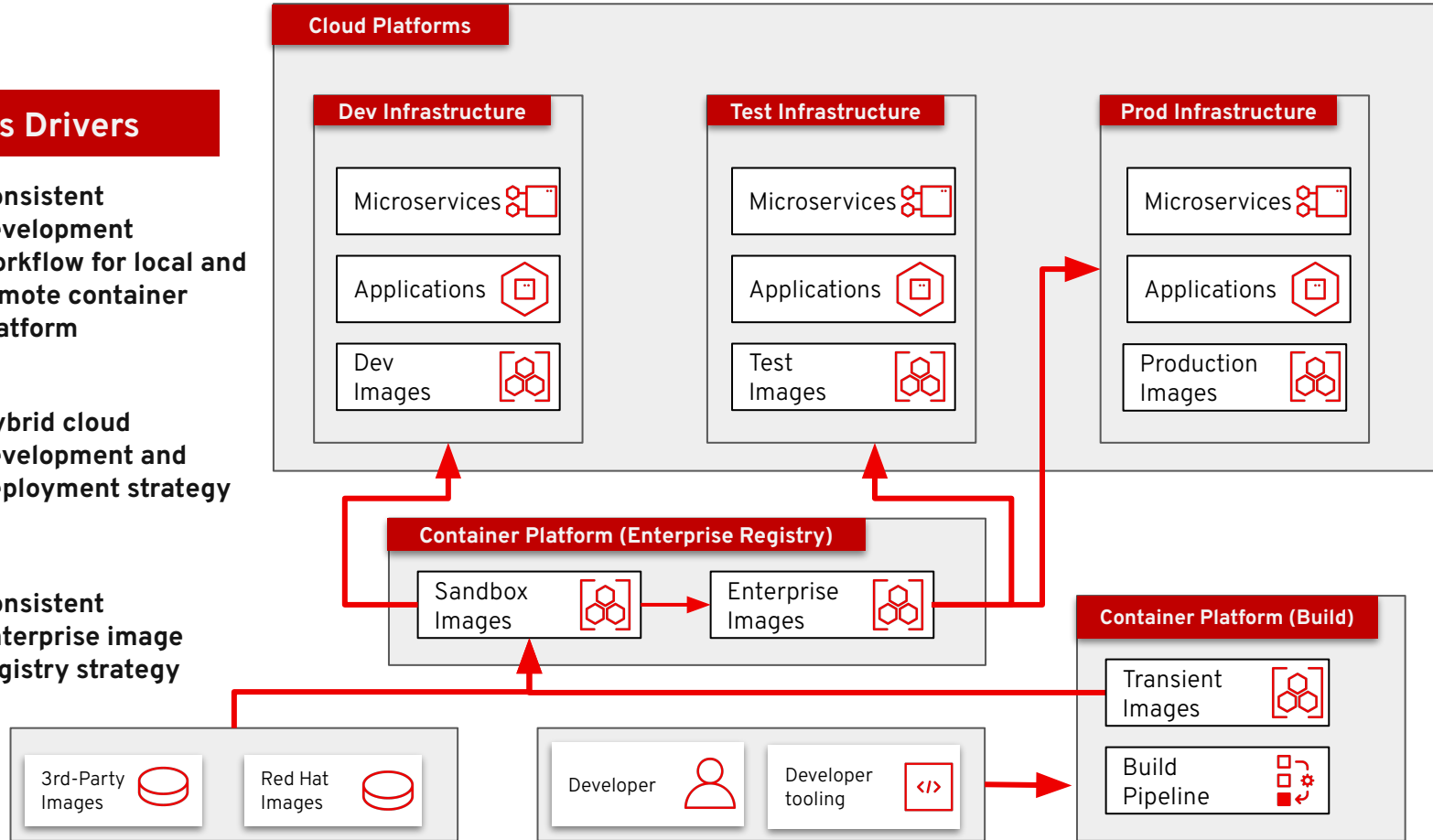
Consistent development workflow for local and remote container platform



Hybrid cloud development and deployment strategy

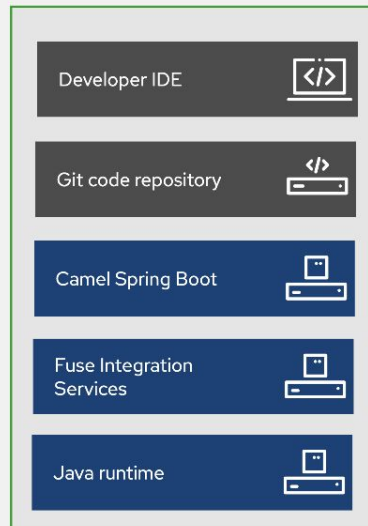


Consistent enterprise image registry strategy



Logical diagram - Cloud-native development

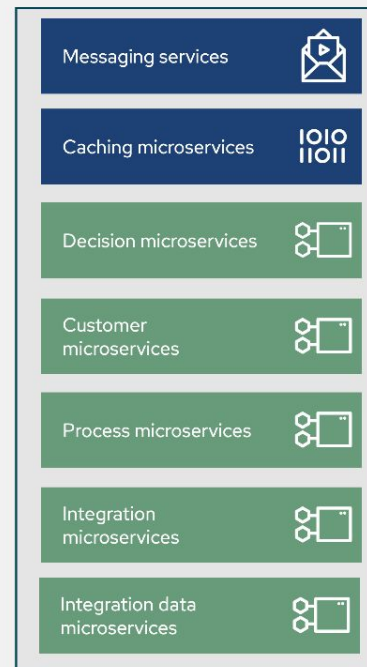
Developer



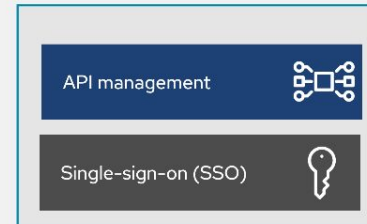
Container platform CI/CD



Container platform



Infrastructure services



Control plane / management

Storage services



Color key

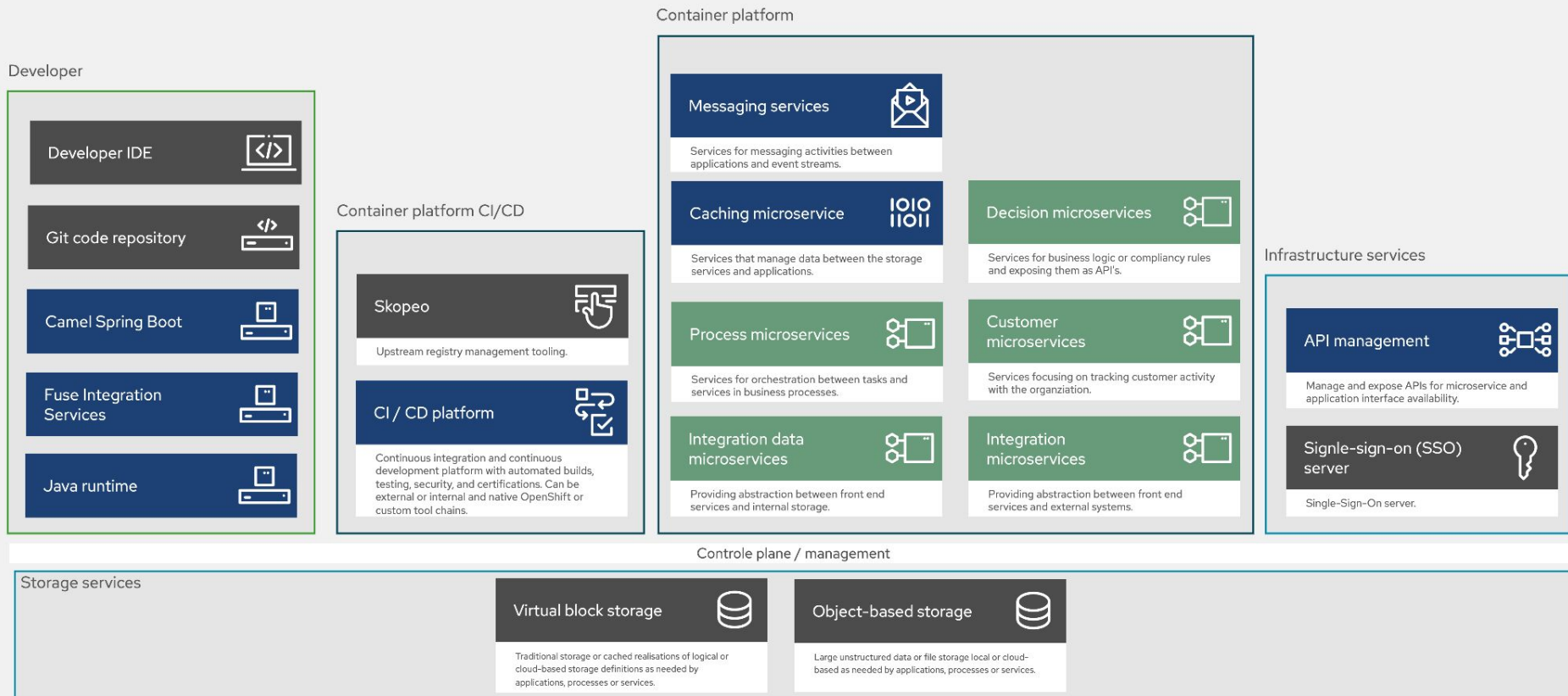
Application

Infrastructure

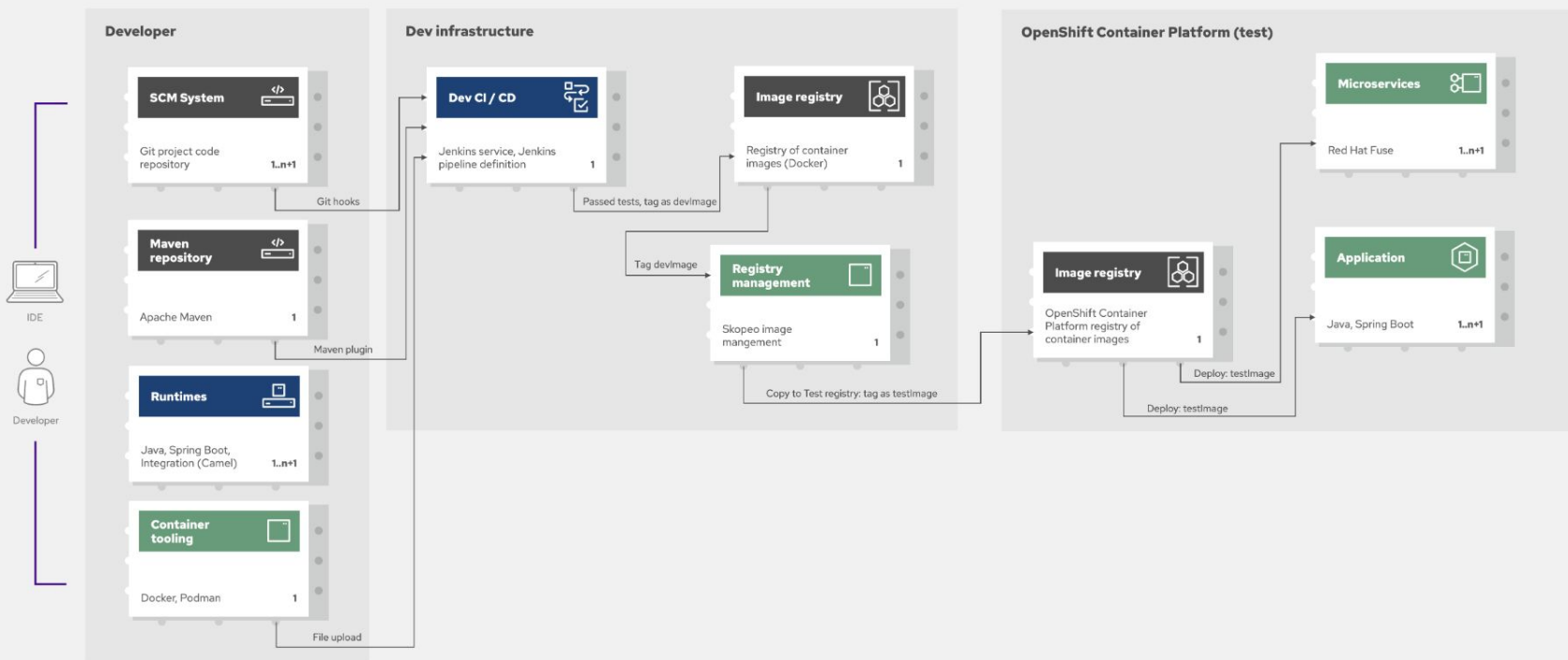
Developer

Services

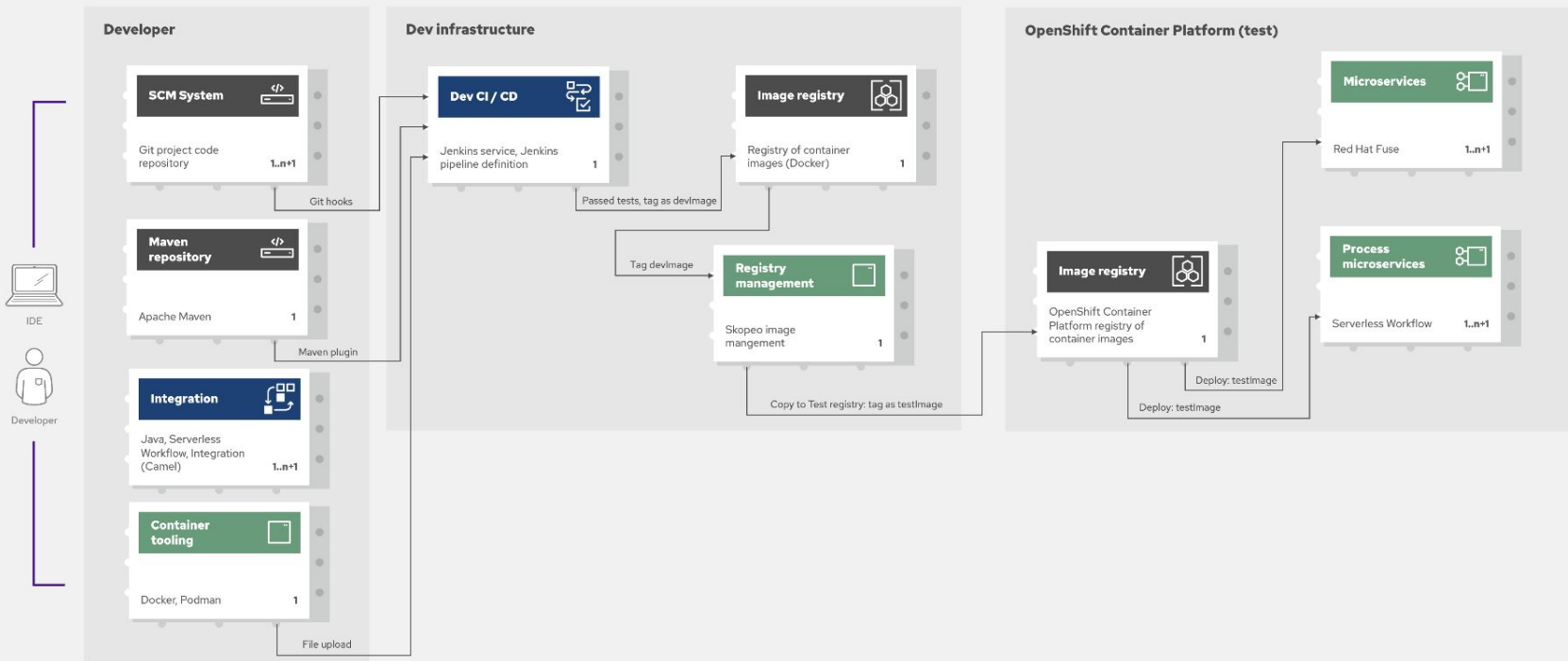
Logical diagram - Cloud-native development (details)



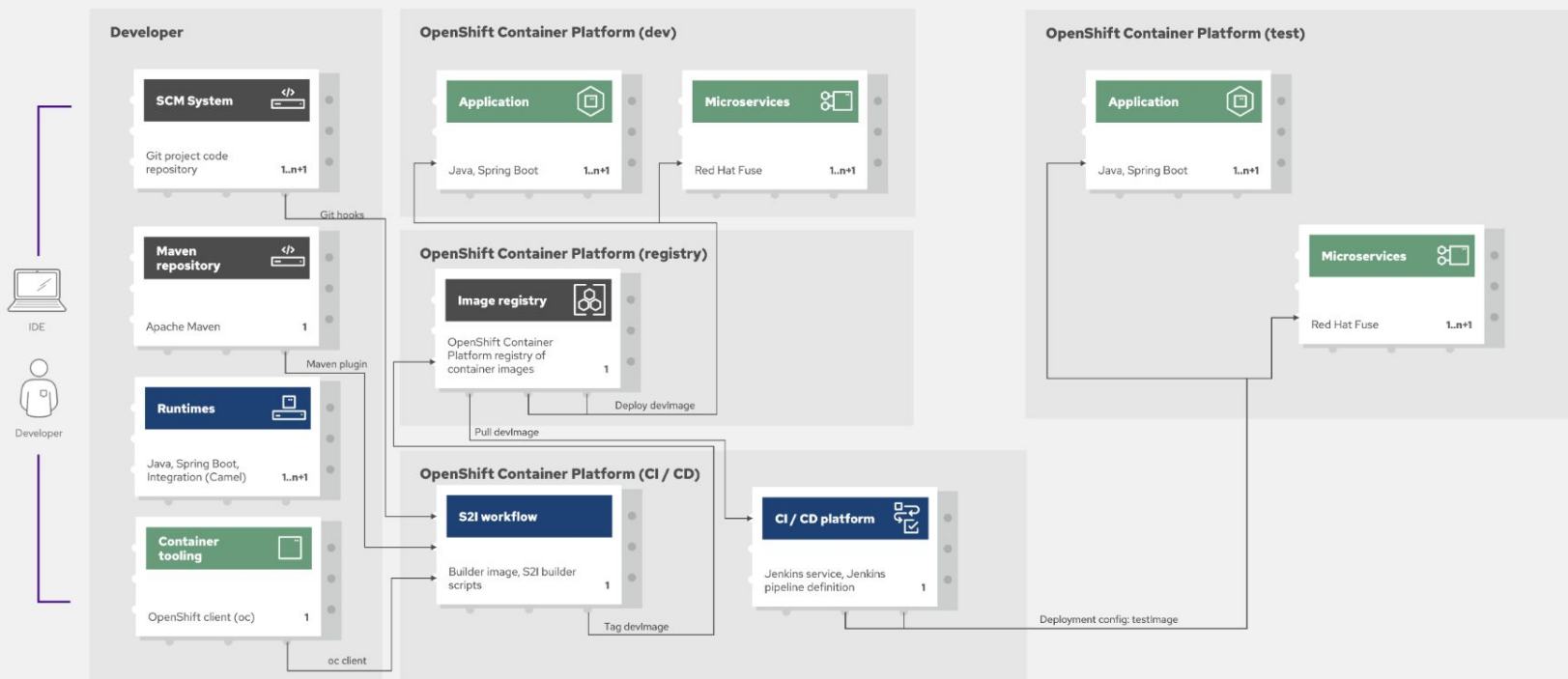
Schematic diagram – cloud-native development with local containers (Runtimes + Integration)



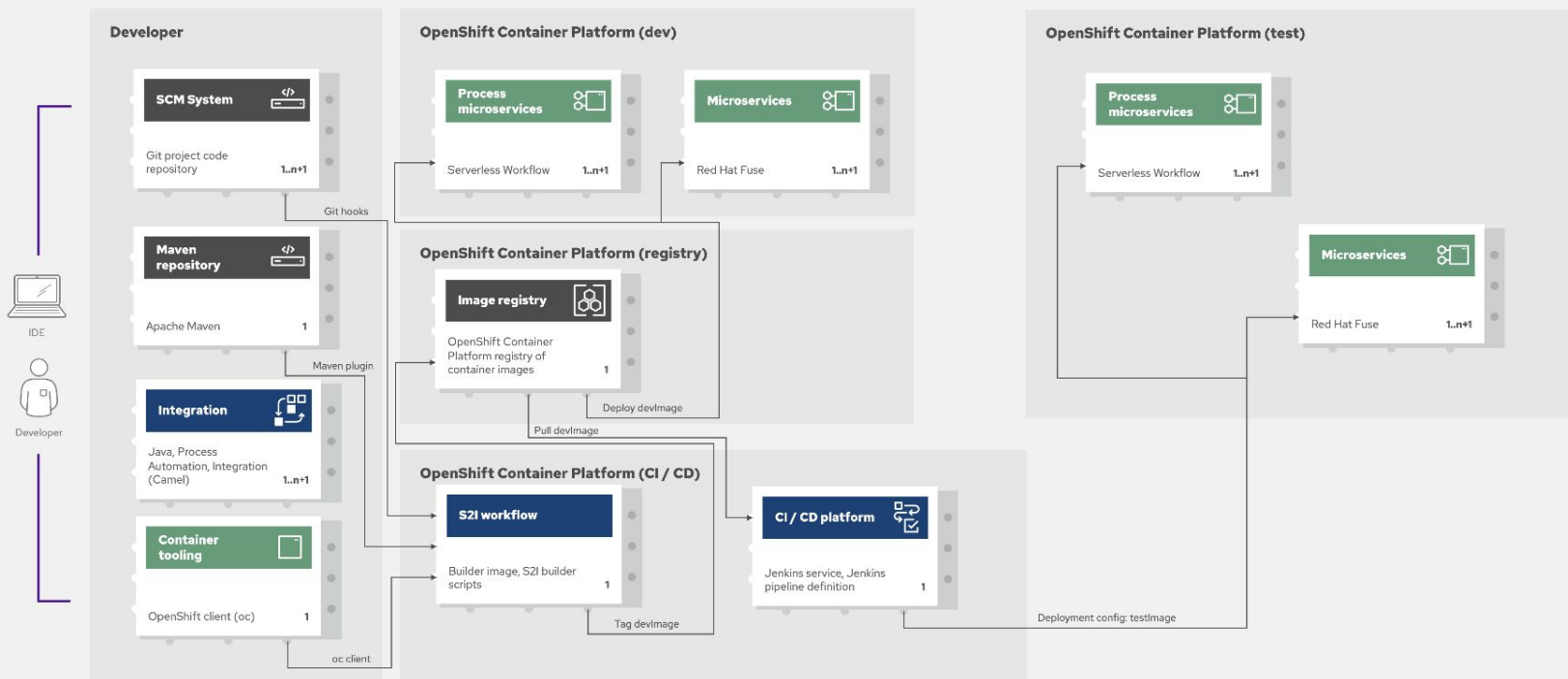
Schematic diagram – cloud-native development with local containers (Process Automation)



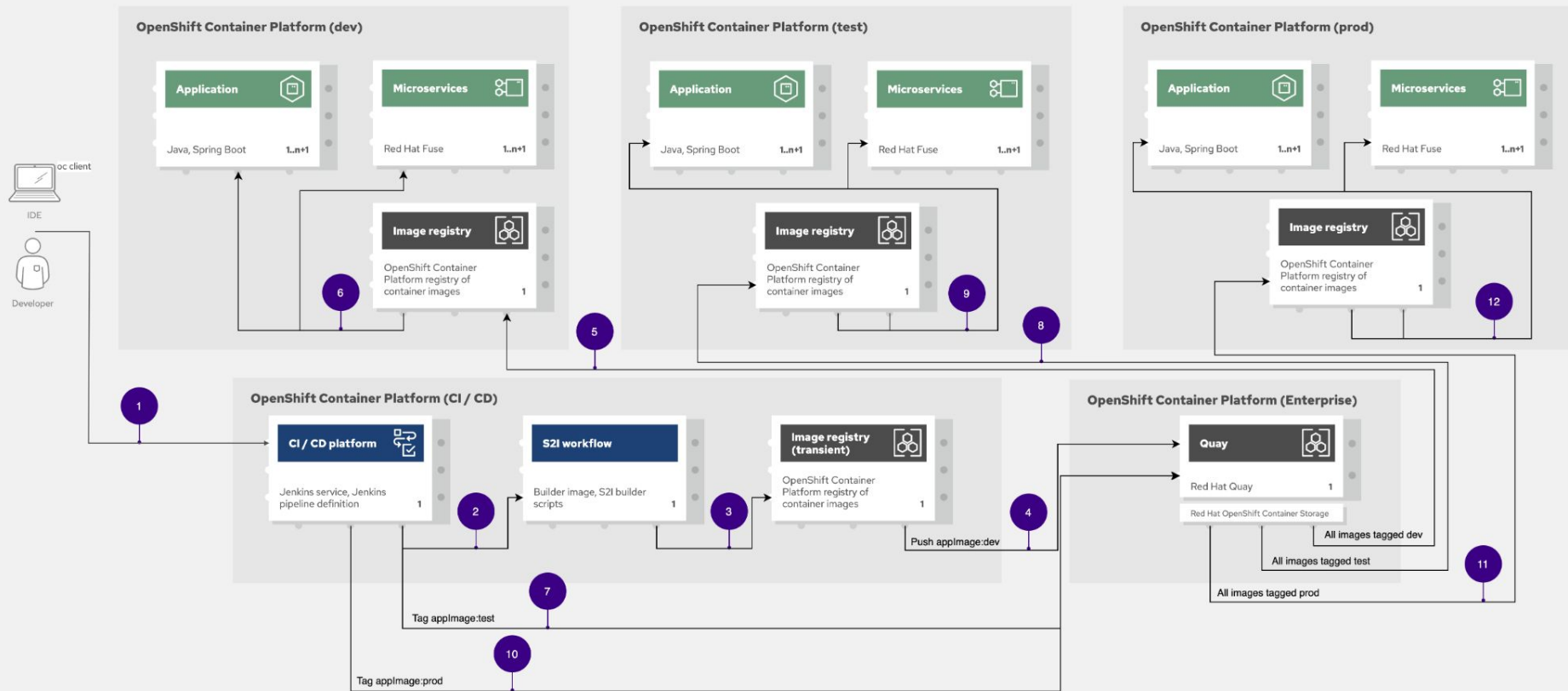
Schematic diagram – cloud-native development with remote containers (Runtimes + Integration)



Schematic diagram – cloud-native development with remote containers (Process Automation)



Schematic diagram - cloud-native development deployment



Color key

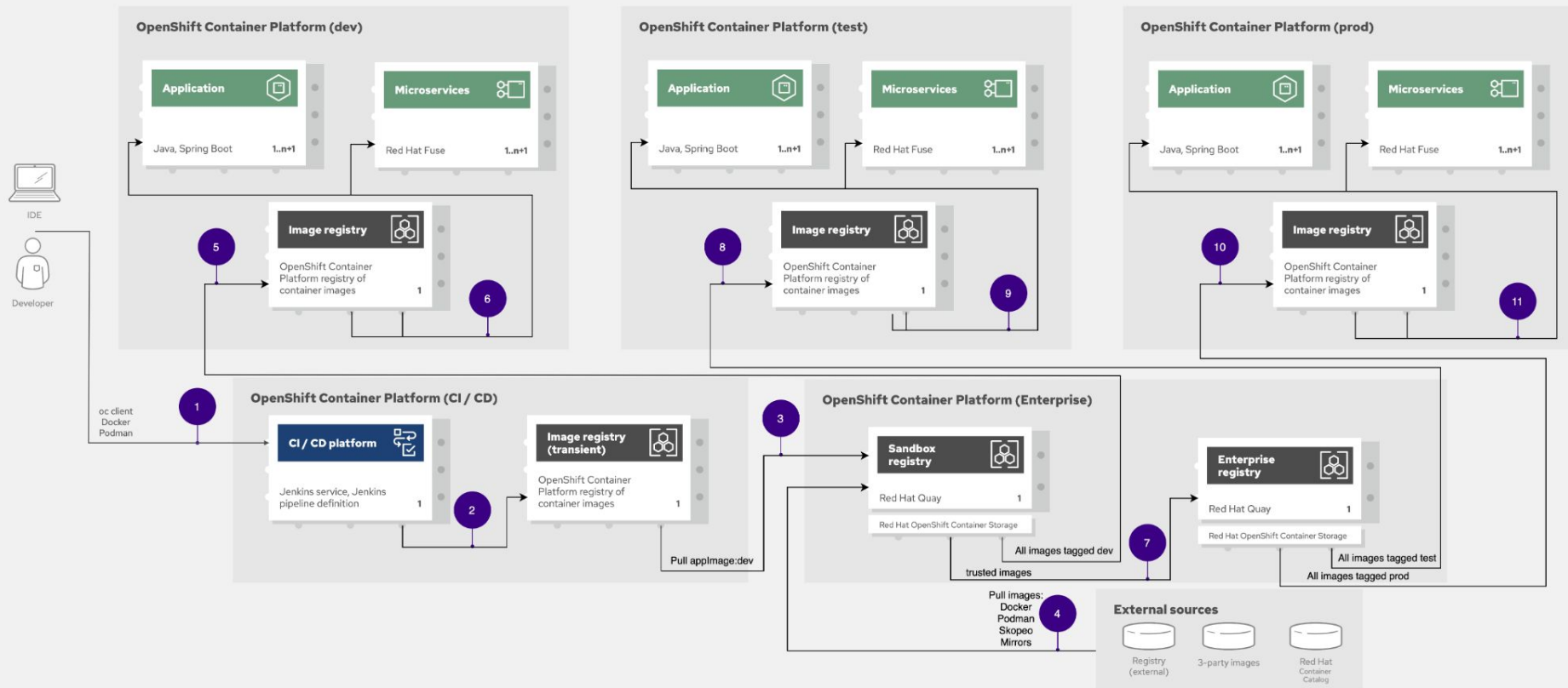
Application

Infrastructure

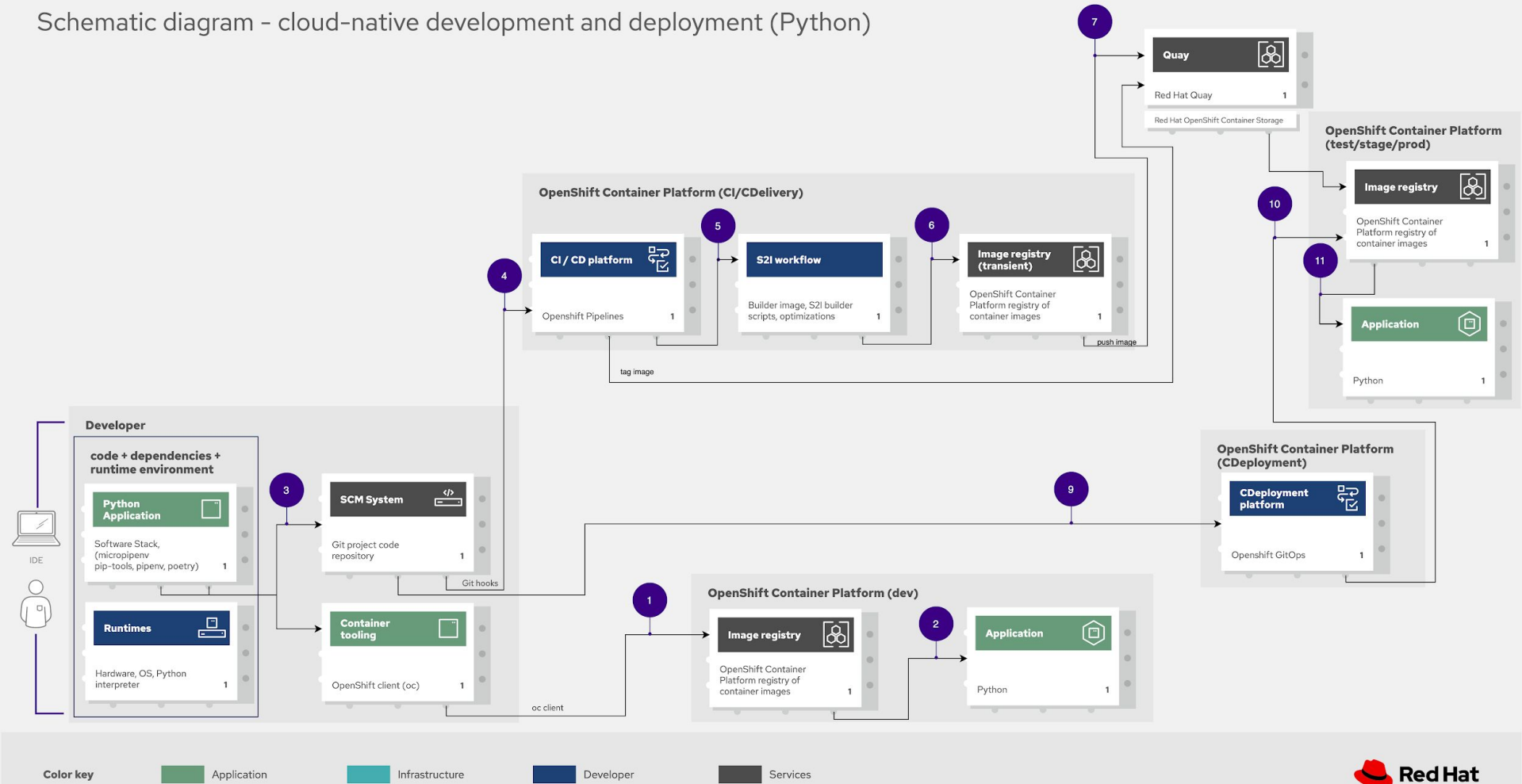
Developer

Services

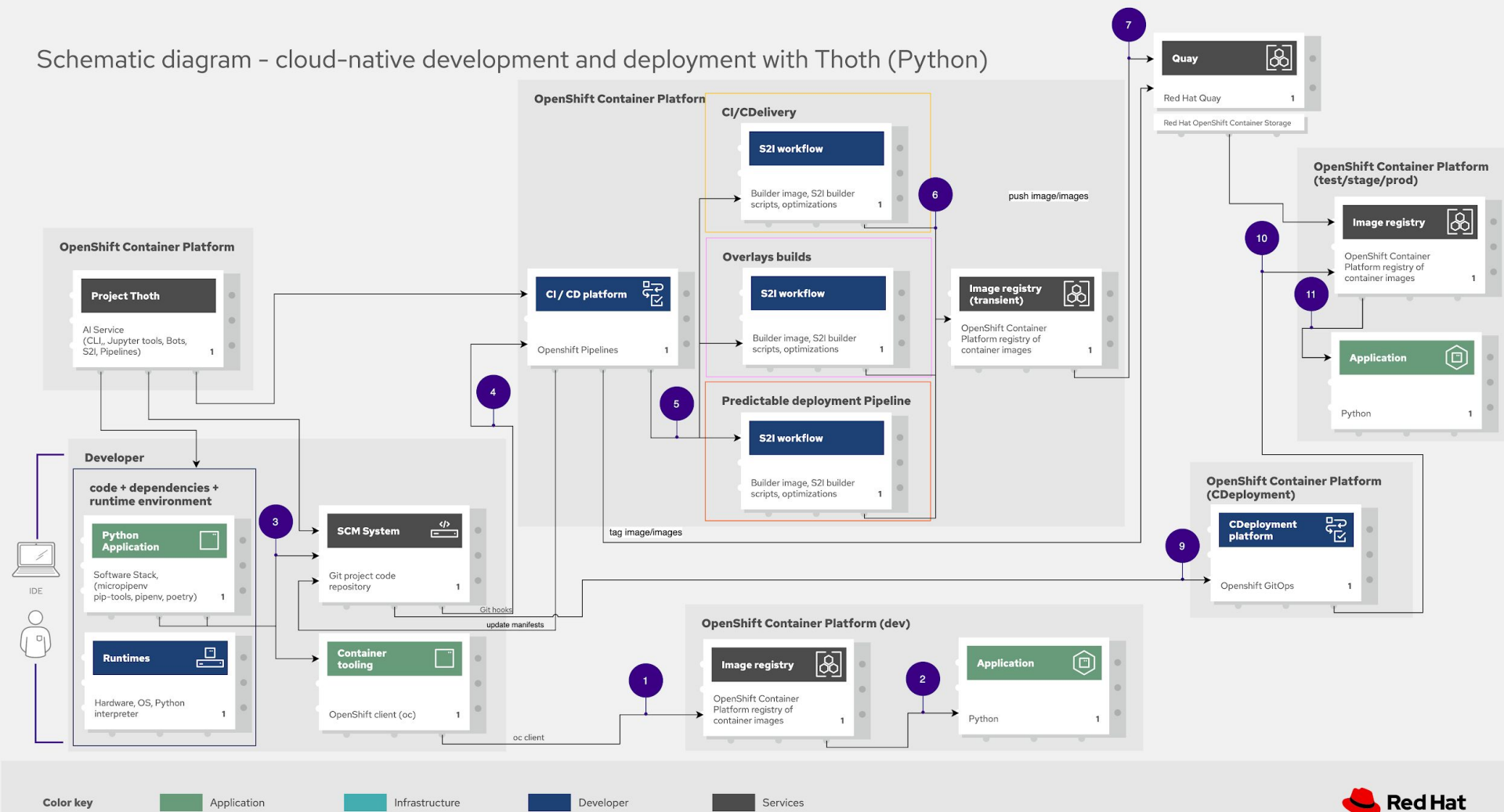
Schematic diagram – cloud-native development deployment (Enterprise registry)

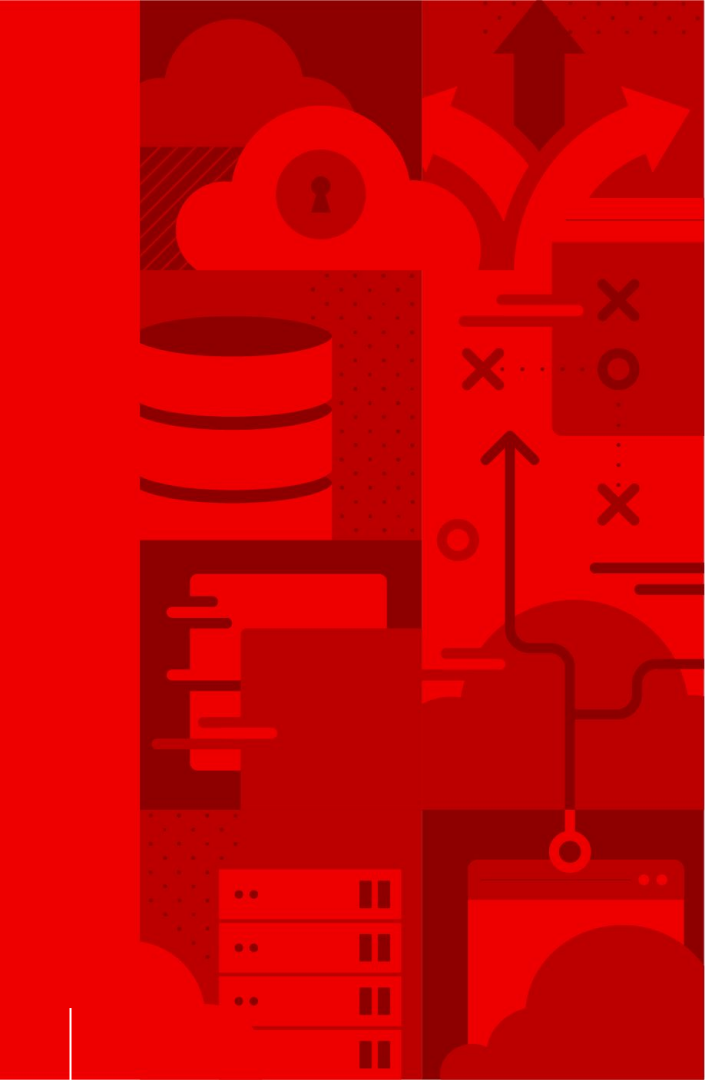


Schematic diagram - cloud-native development and deployment (Python)



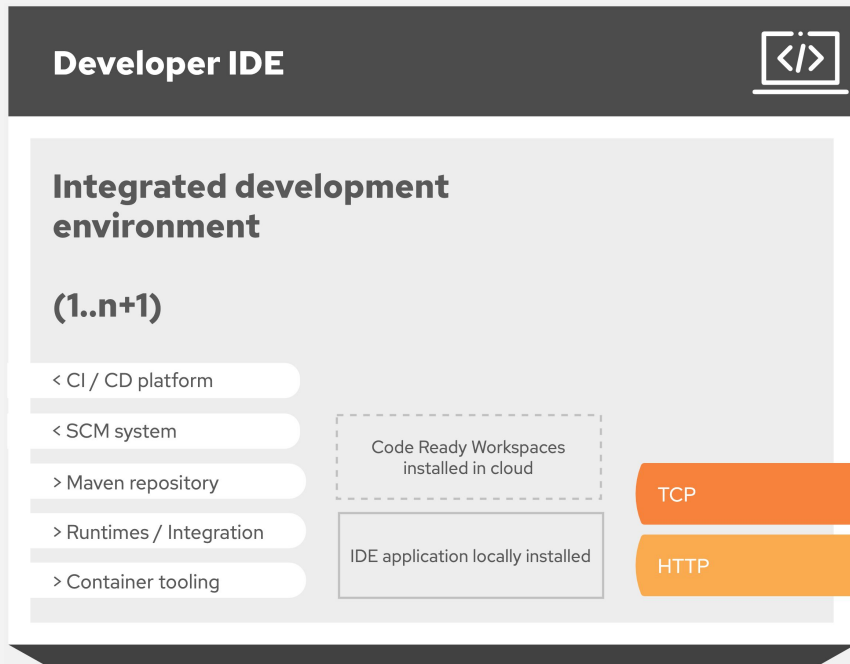
Schematic diagram - cloud-native development and deployment with Thoth (Python)





APPENDIX WITH ELEMENT DETAILS

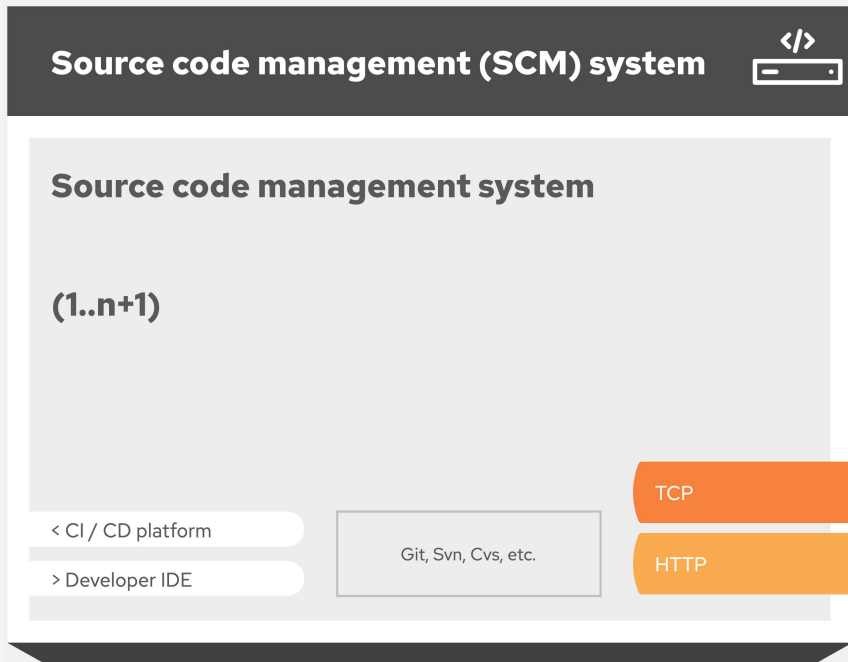
DETAIL: DEVELOPER IDE



Integrated development environment for code production:

- Local developer IDE
- Remote cloud-based developer IDE
- Pulls from:
 - Runtimes / Integration frameworks
 - Maven repository
 - Container tooling
- Pushes to:
 - SCM system
 - CI / CD platform

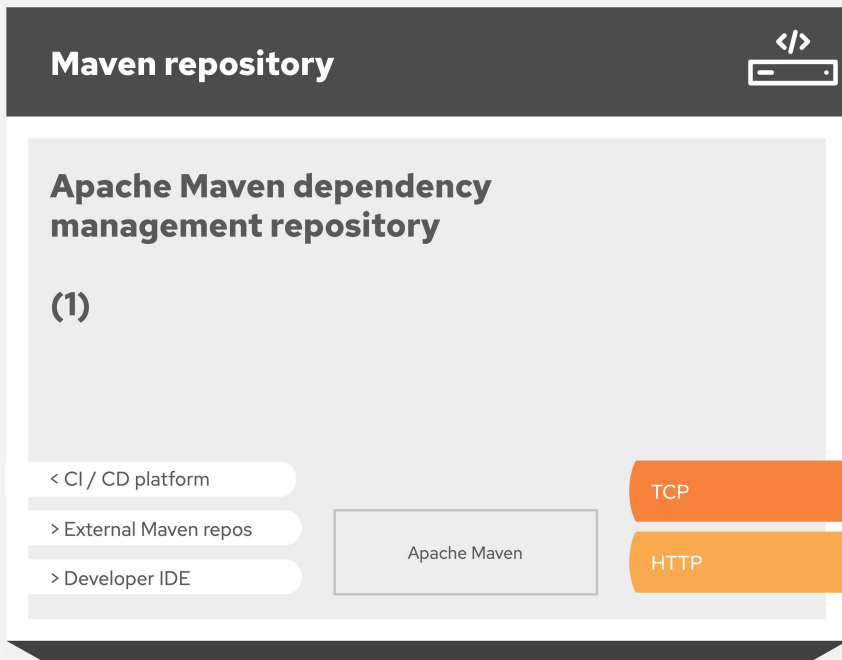
DETAIL: SCM SYSTEM



Source code management system:

- Use for cross developer code production and management
- Input from:
 - Developer IDE
- Output to:
 - CI / CD platform

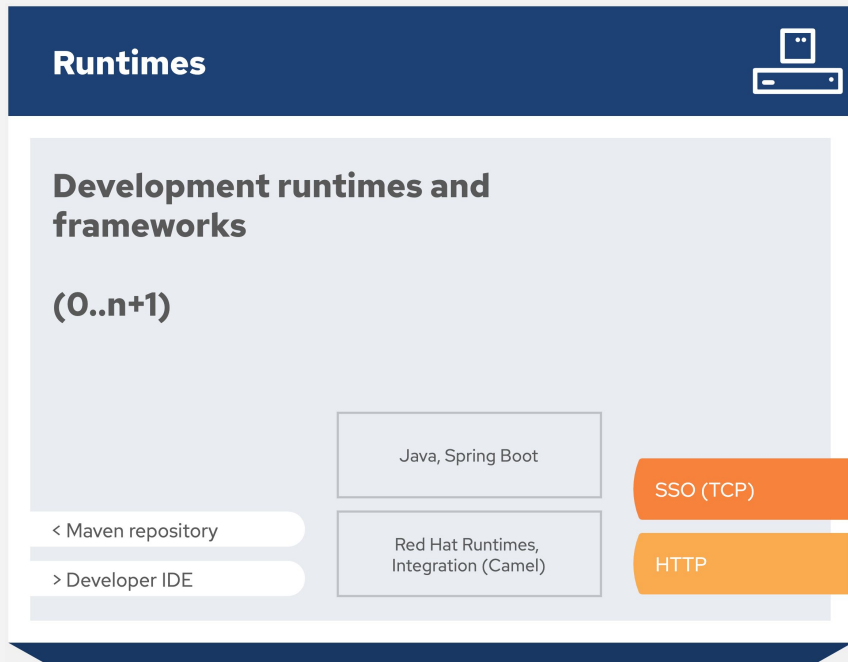
DETAIL: MAVEN REPOSITORY



Dependency management system using Apache Maven:

- Maintains developer dependencies for projects
- Input from:
 - Developer IDE
 - External Maven repositories
- Output to:
 - CI / CD platform

DETAIL: RUNTIMES



Developer runtimes and development frameworks:

- Java, Spring Boot, Integration (Camel), etc.
- Supported with Red Hat Runtimes
- Input from:
 - Developer IDE
- Output to:
 - Maven repository

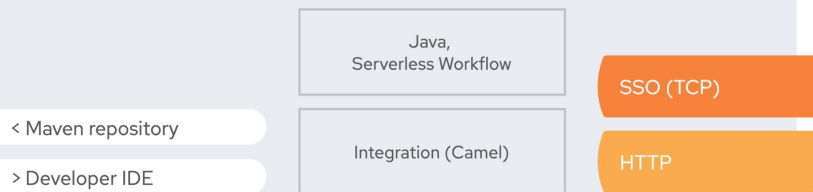
DETAIL: INTEGRATION

Integration



Development integration and process automation

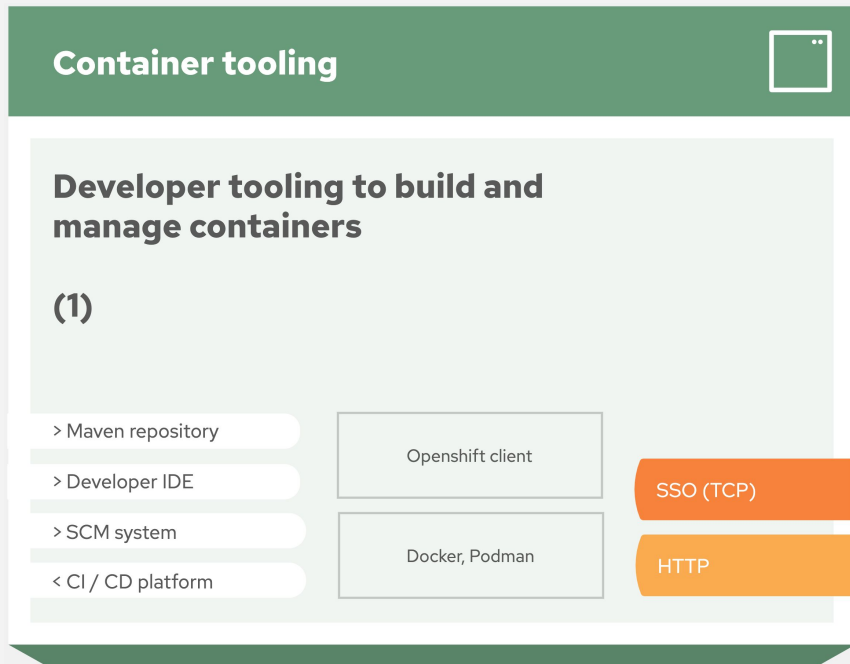
(0..n+1)



Developer integration and process automation development frameworks:

- Java, Integration (Camel)
- Serverless Workflow
- Input from:
 - Developer IDE
- Output to:
 - Maven repository

DETAIL: CONTAINER TOOLING



Developer local tooling for container building and management.

- Docker and Podman tooling
- OpenShift client tooling
- Input from:
 - Developer IDE
 - Maven repository
 - SCM system
- Output to:
 - CI / CD platform

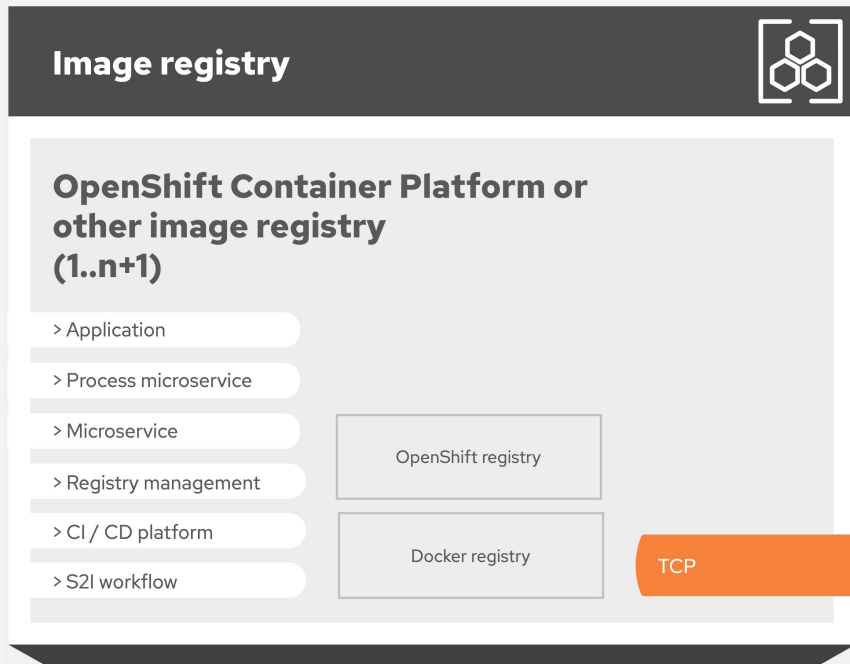
DETAIL: CI / CD PLATFORM



Continuous integration and development platform for builds and testing development pipeline:

- OpenShift Pipelines, Jenkins, etc.
- Automation of testing and tagging images
- Input from:
 - Container tooling
 - Maven repository
 - SCM system
- Output to:
 - S2I workflow
 - Image repository

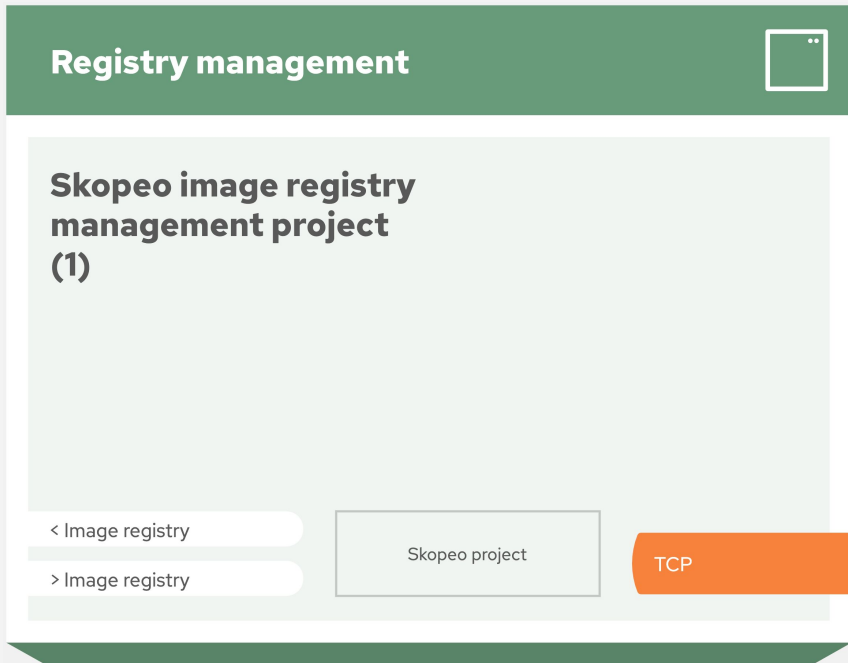
DETAIL: IMAGE REGISTRY



The image registry as provided by OpenShift Container Platform or a docker image registry:

- Docker, OpenShift, etc.
- Registry of images for deployment
- Input from:
 - CI / CD platform
 - S2I workflow
- Output to:
 - Registry management
 - Microservice
 - Application
 - Process microservice

DETAIL: REGISTRY MANAGEMENT



Application to manage image copies between registries:

- Skopeo project
- Input from:
 - Image registry
- Output to:
 - Image registry

DETAIL: S2I WORKFLOW

S2I workflow

Predefined workflow building container image from source code

(1..n+1)

< Image registry

> CI / CD platform

> Container tooling

OpenShift client (oc)

Podman

Docker

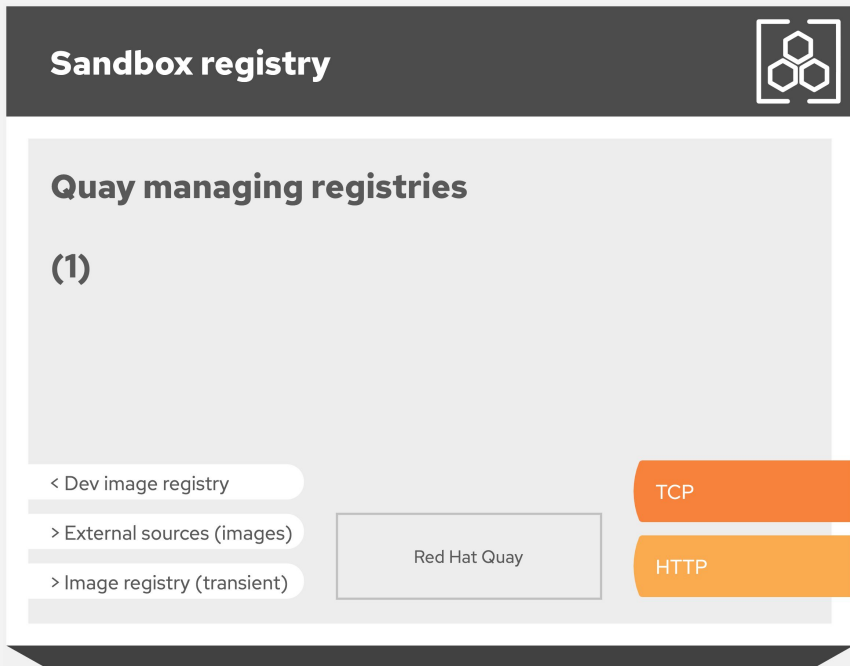
TCP

HTTP

Source to image workflow to build container image from code:

- Often customer defined workflow to container image
- Input from:
 - CI / CD platform
 - Container tooling
- Output to:
 - Image registry

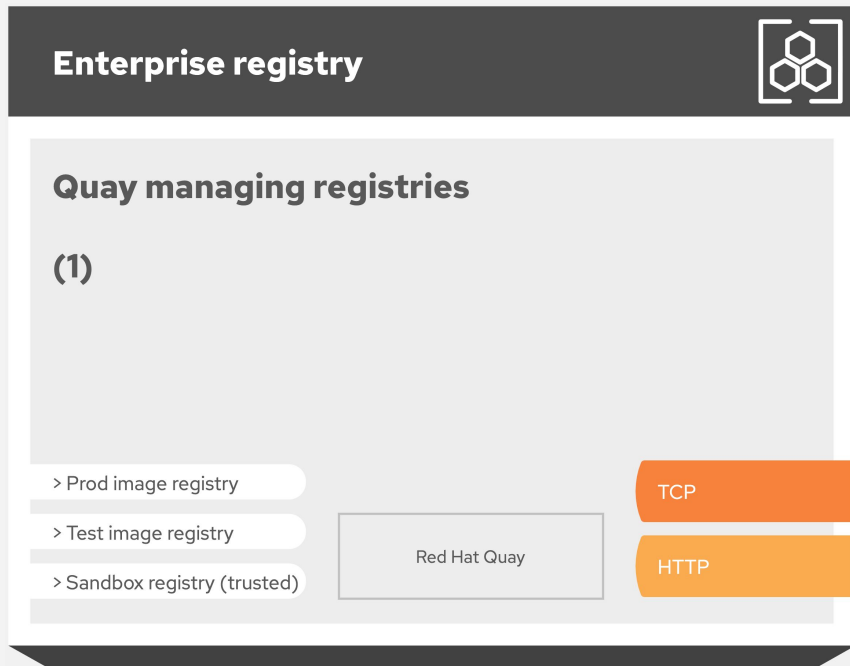
DETAIL: SANDBOX REGISTRY



OpenShift Container Platform image registry managing untrusted images:

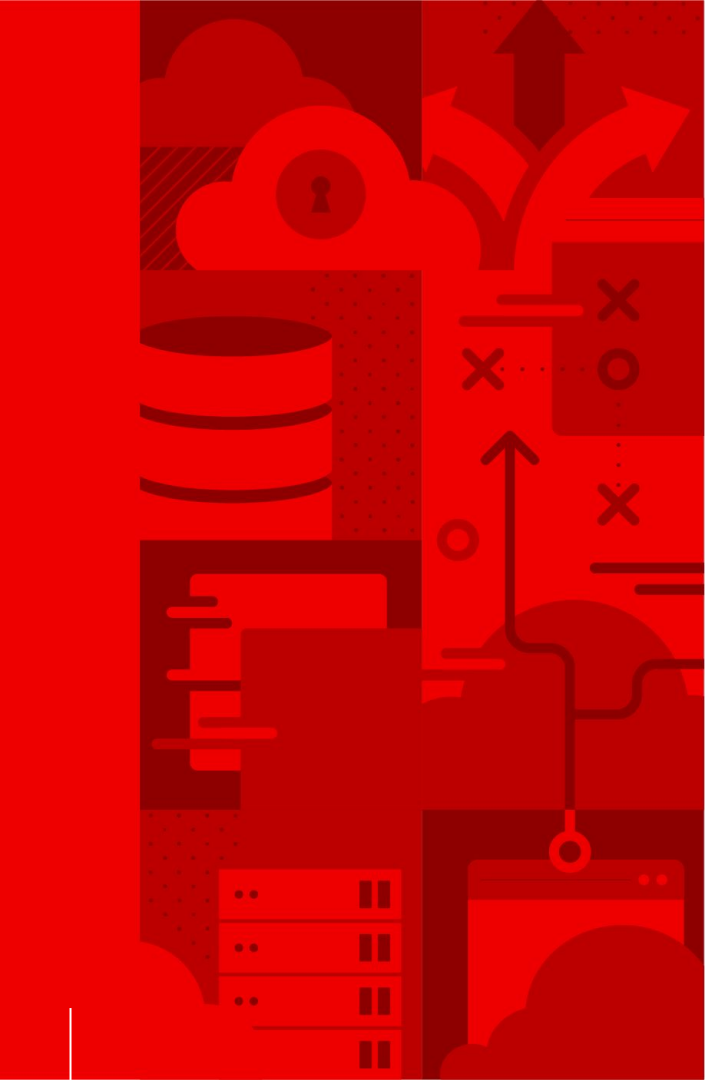
- Red Hat Quay managed registry
- Untrusted and dev images in registry
- Input from:
 - Image registry (transient)
 - External sources (untrusted images)
- Output to:
 - Dev image registry
 - Enterprise registry

DETAIL: ENTERPRISE REGISTRY



OpenShift Container Platform image registry managing trusted images:

- Red Hat Quay managed registry
- Trusted images in registry
- Input from:
 - Sandbox registry (trusted)
- Output to:
 - Test image registry
 - Production image registry



Thank You

End of the public story